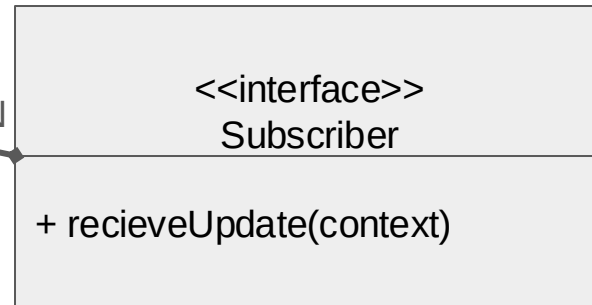


CSC1031

OOP



0..N

StockMarket

Venture
Capital

CryptoMarket

APL

IBM

TSL

BTC

ETH

DGE

Trader

Company

Jhon

Eva

A client

Java Program Structure

Components:

- **Package:** Defines a namespace to organize code.
- **Class:** The blueprint for objects.
- **Methods:** Perform actions in a class.
- **Main Method:** The entry point of a Java application.

Basic Java Program Structure

`package com.example;` → Groups related classes.

`public class Main` → Defines a class.

`public static void main(String[] args)` → The main method, execution starts here.

```
package com.example; // Package declaration (optional)

public class Main { // Class declaration
    public static void main(String[] args) { // Entry point
        System.out.println("Hello, Java!");
    }
}
```

What is a Package in Java?

- A package is a way to group related Java classes together.
- It provides organization, reusability, and access control.
- Packages help prevent name conflicts between classes.

Encapsulation – Controls access to classes.

Code Organization – Helps manage large codebases.

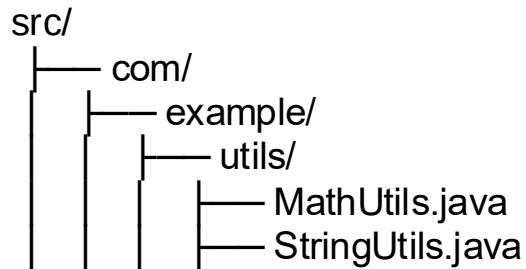
Avoids **Naming Conflicts** – Prevents different libraries from having the same class names.

Modular Development – Makes maintenance and collaboration easier.

!!! Each **class in the same folder should have the same package** declaration. !!!

Package Structure

When you define a package, Java organizes files in directories



com.example.utils corresponds to the folder com/example/utils/

Java uses dot notation (.) to represent folder hierarchy

```
package com.example; // Package declaration (optional)

public class Main { // Class declaration
    public static void main(String[] args) { // Entry point
        System.out.println("Hello, Java!");
    }
}
```

How to Use a Package?

```
javac -d . com/example/Utils/MathUtils.java com/example/Main.java
```

```
java com.example.Main
```

```
import com.example.Utils.MathUtils; // Importing a class

public class Main {
    public static void main(String[] args) {
        int sum = MathUtils.add(5, 3);
        System.out.println("Sum: " + sum);
    }
}
```

Java Built-in Packages

Java has many standard packages:

- **java.lang** (Automatically imported)
 - `String`, `Math`, `System`
- **java.util** (Utility classes)
 - `List`, `ArrayList`, `HashMap`
- **java.io** (File handling)
 - `File`, `BufferedReader`
- **java.net** (Networking)
 - `URL`, `Socket`
- **java.sql** (Database connectivity)
 - `Connection`, `Statement`

Using Build Tools

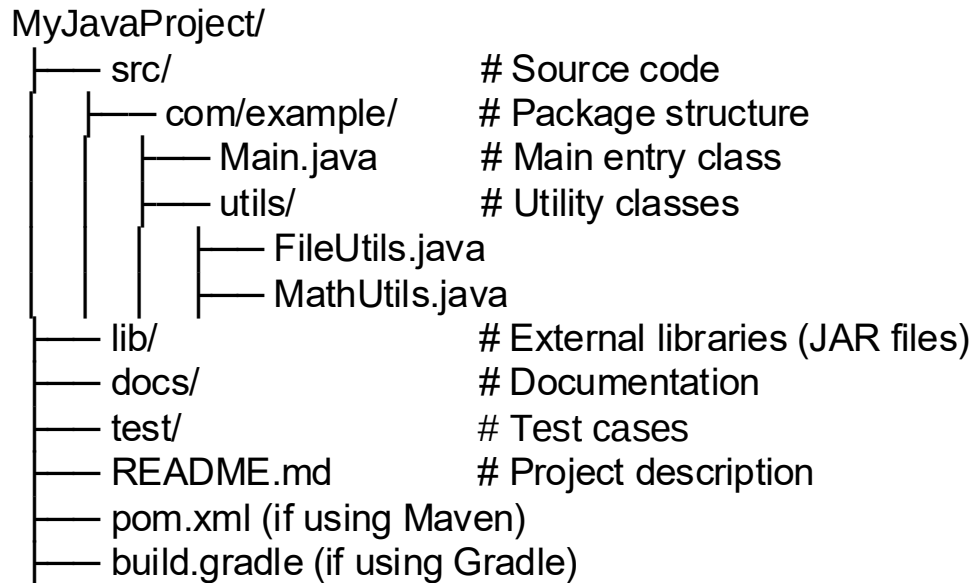
Maven (Dependency Management)

- Creates a project structure automatically.
- Handles external dependencies.
- Uses XML

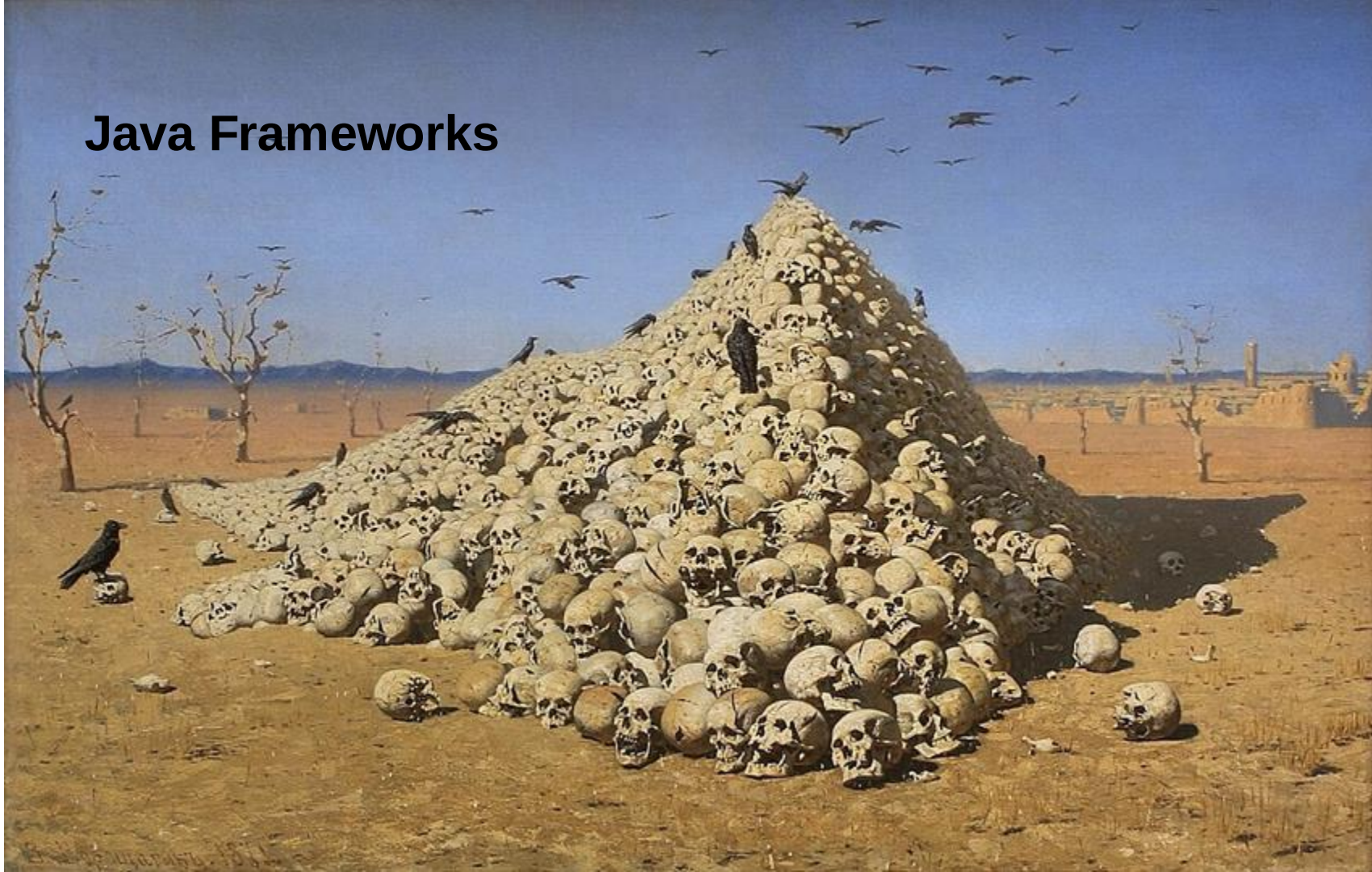
Gradle (Alternative to Maven)

- Uses Groovy or Kotlin scripts instead of XML.

Java Project Organization



Java Frameworks



Why Use Frameworks?

Speeds up development

Encourages best practices

Reduces code duplication

Provides built-in functionalities

Hibernate

an **ORM** (Object-Relational Mapping) framework for managing databases in Java

Converts Java objects into database records

Supports HQL (Hibernate Query Language)

Works with multiple databases (MySQL, PostgreSQL, etc.)



Spring Framework

the most widely used enterprise-level framework for building Java applications.

Spring Boot: Simplifies application development

Spring MVC: Supports web applications

Spring Data: Simplifies database access

Spring Security: Manages authentication and authorization

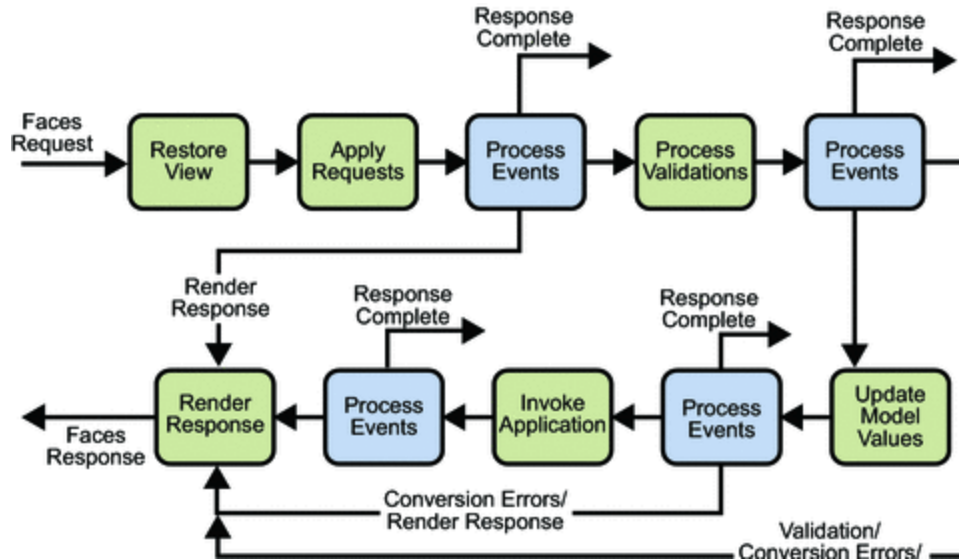


JavaServer Faces (JSF)

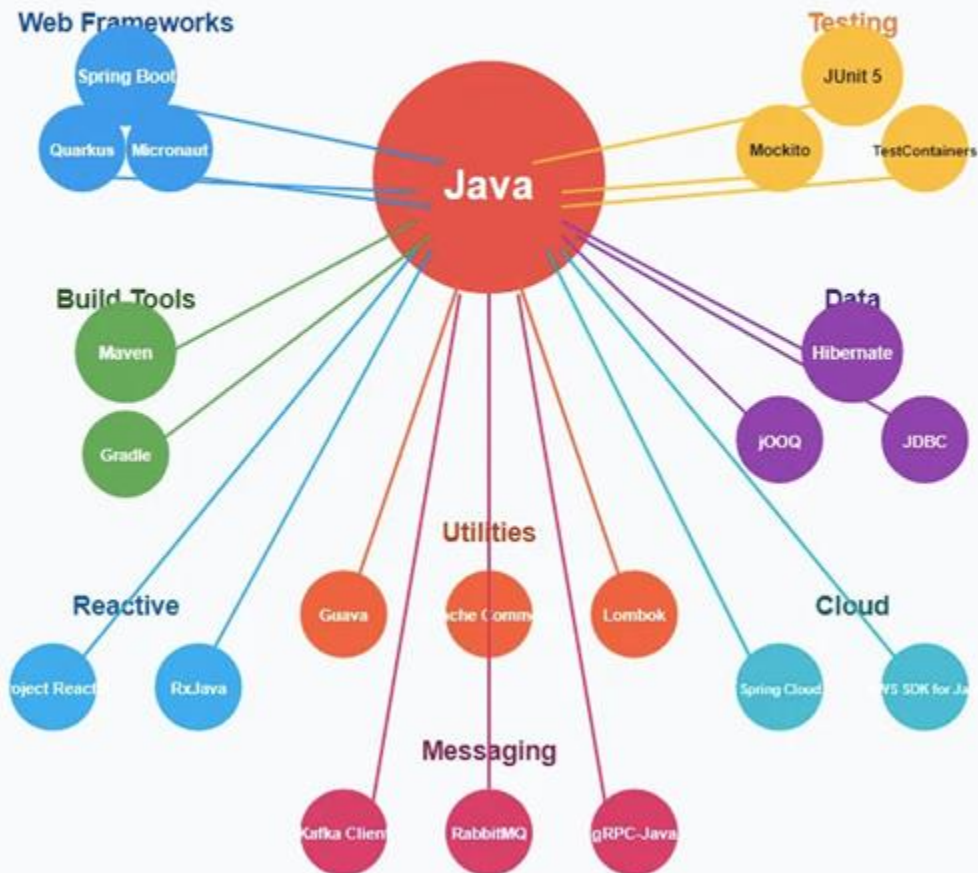
a Java web framework for building user interfaces

Uses XML-based templates for UI

Supports event handling and component-based architecture



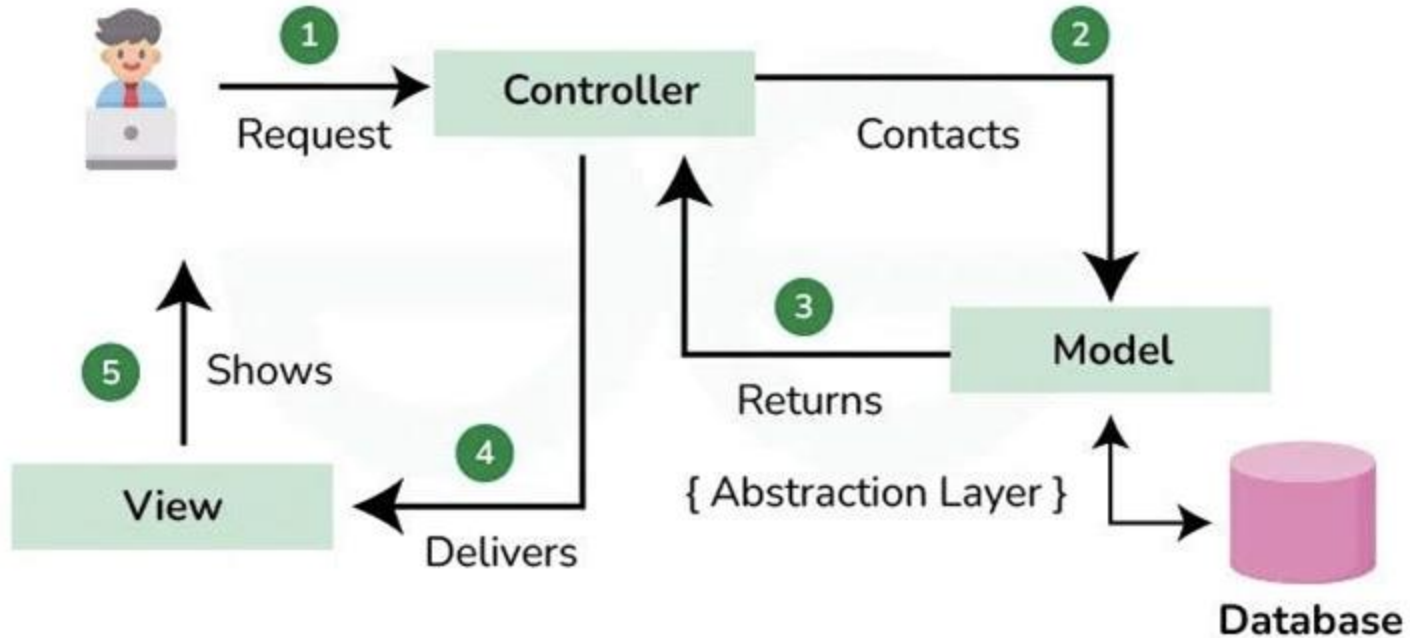
22 Essential Java Libraries for 2025



Framework vs library

Definition	A collection of pre-written code that developers can call to perform specific tasks.	A complete structure for building applications, enforcing a specific way to write code.
Control Flow	You call the library methods when needed.	The framework controls when and how your code is executed (Inversion of Control - IoC).
Flexibility	You can use only what you need, mix with other libraries.	You must follow the framework's structure and conventions.
Example	Apache Commons (String manipulation, file handling)	Spring Framework (Enforces MVC structure for web apps)
Who is in control?	Developer decides when and how to use it.	Framework dictates how the code should be structured and executed.

MVC architecture



Adapter



Adapter Pattern

The Adapter Pattern is a **structural design pattern** that allows two incompatible interfaces to work together.

Commonly used when integrating third-party libraries or legacy code into modern applications.

Also Known As: **Wrapper Pattern**



Problem

A stock market app downloads XML-based stock data from multiple sources.

A third-party analytics library is introduced to enhance the app, but it only supports JSON data.

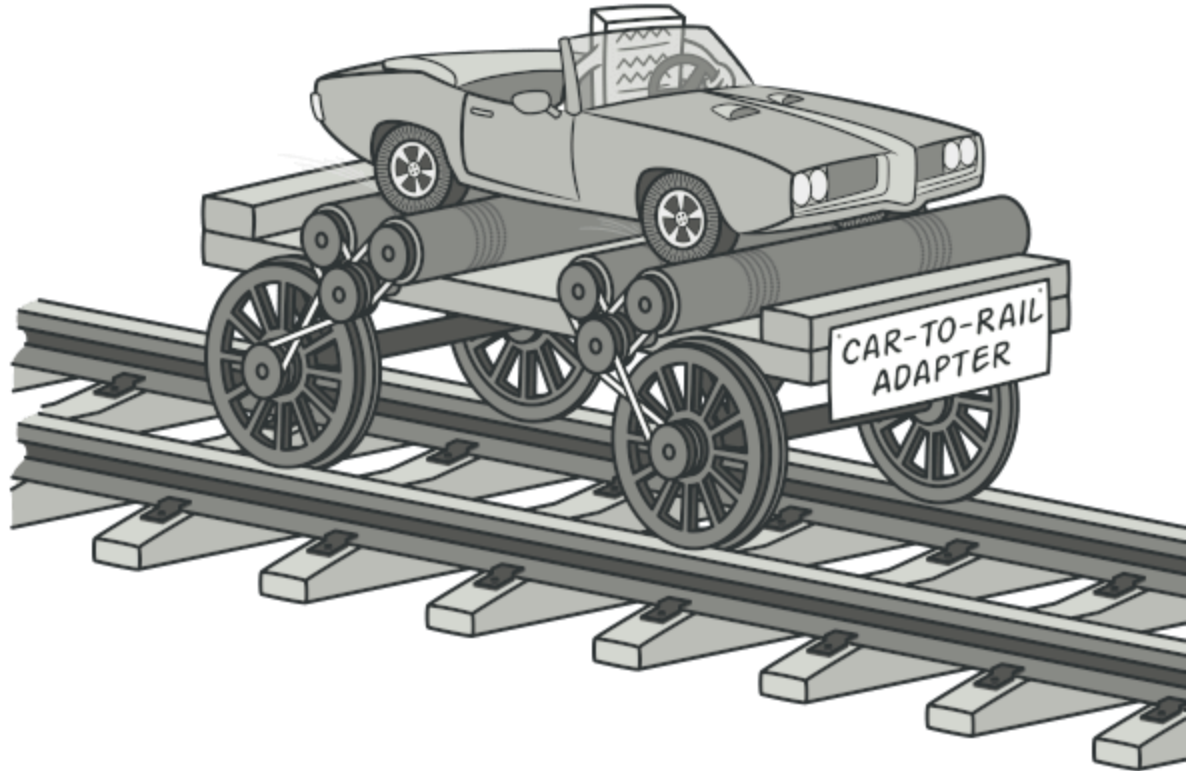
Directly modifying the library to accept XML is risky (may break existing code) or impossible (source code unavailable).

Solution

Allows seamless integration of new features without modifying existing systems.

✓ Decouples the app from the third-party library, making it flexible for future changes.

✓ Follows the Open/Closed Principle – adapts behavior without modifying existing classes.



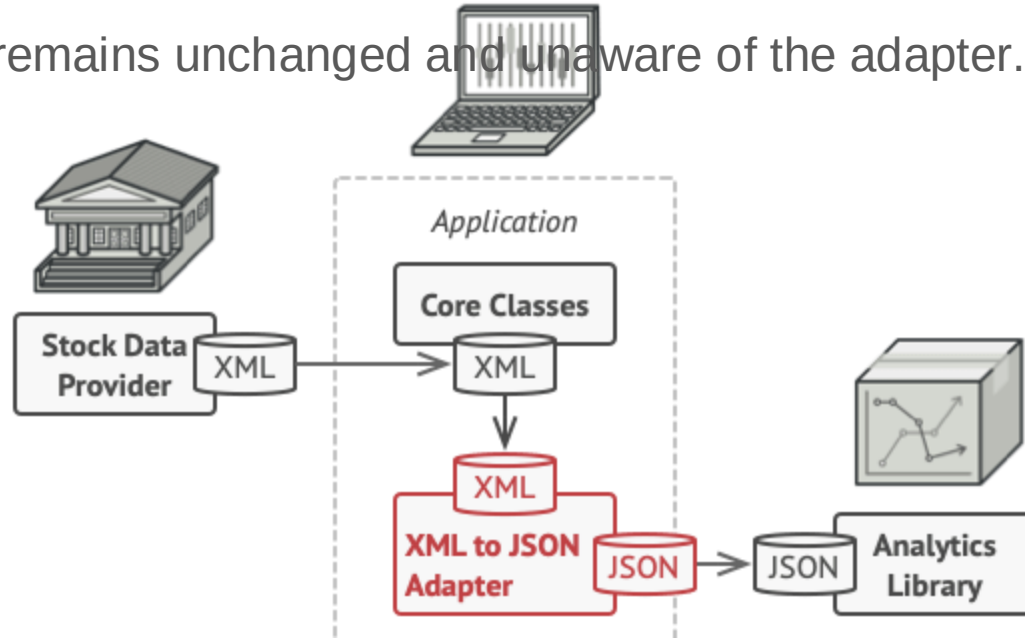
How it works

The adapter wraps the XML-based system.

It converts XML to JSON behind the scenes.

The analytics library receives JSON as expected.

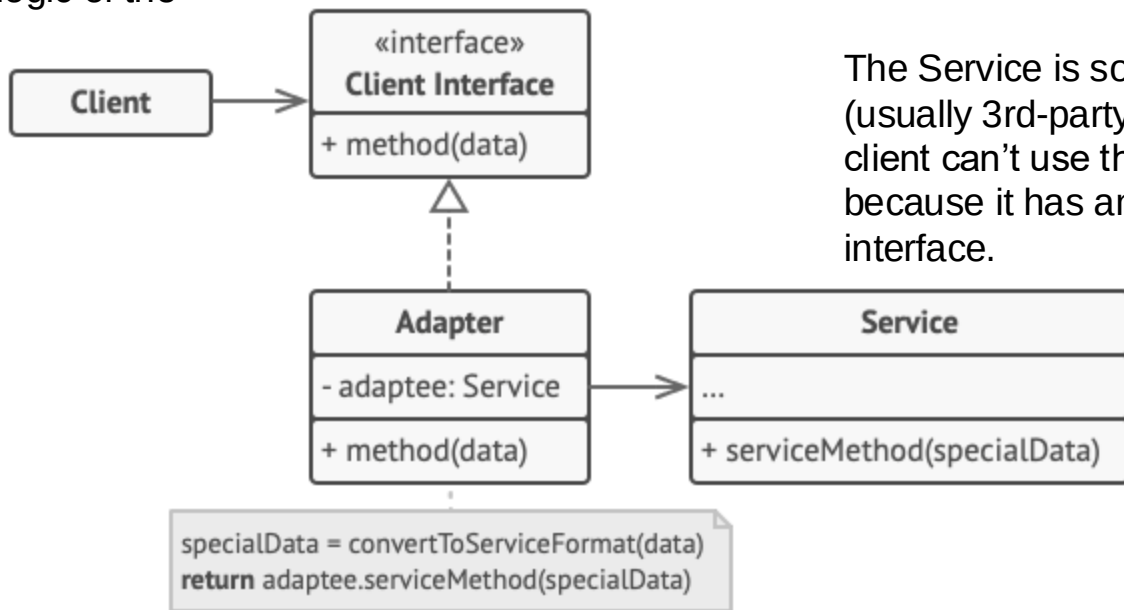
The original system remains unchanged and unaware of the adapter.



Structure

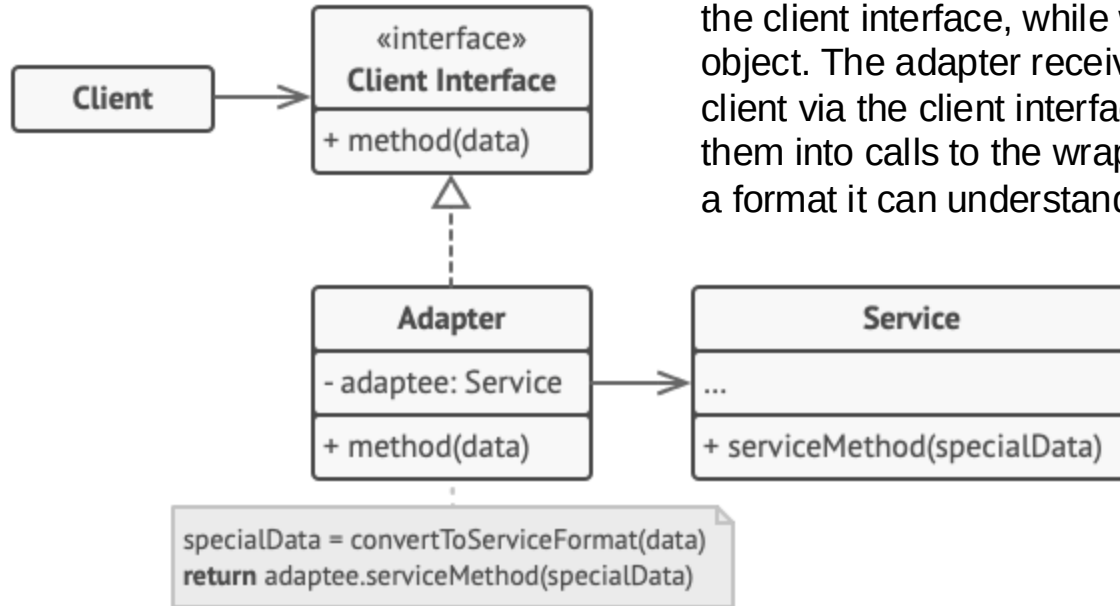
The Client is a class that contains the existing business logic of the program.

The Client Interface describes a protocol that other classes must follow to be able to collaborate with the client code.



The Service is some useful class (usually 3rd-party or legacy). The client can't use this class directly because it has an incompatible interface.

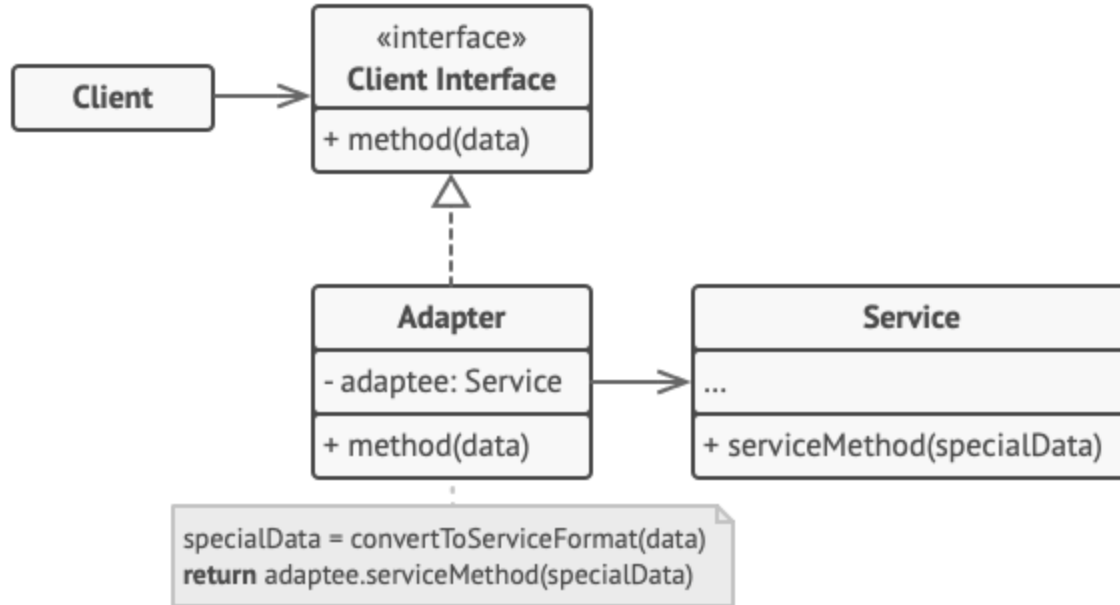
Structure



The Adapter is a class that's able to work with both the client and the service: it implements the client interface, while wrapping the service object. The adapter receives calls from the client via the client interface and translates them into calls to the wrapped service object in a format it can understand.

Structure

The client code doesn't get coupled to the concrete adapter class as long as it works with the adapter via the client interface. Thanks to this, you can introduce new types of adapters into the program without breaking the existing client code. This can be useful when the interface of the service class gets changed or replaced: you can just create a new adapter class without changing the client code.



When to Use the Adapter Pattern?

You need to integrate an existing class that has an incompatible interface with your system.

You're working with legacy code or third-party libraries that cannot be modified.

You want to reuse multiple subclasses that lack common functionality without modifying the superclass.

How to Implement

1. Identify Incompatible Classes

- **Service Class** (unchangeable) – A third-party, legacy, or dependency-heavy class.
- **Client Class** – Needs to use the service class but has an incompatible interface.

1. Define a Common Interface

- Create a client-friendly interface that defines how the client expects to communicate with the service.

1. Create the Adapter Class

- Make it implement the client interface (methods initially empty).
- Store a reference to the service class in a field, initialized via the constructor.

1. Implement the Adapter Methods

- Delegate most of the work to the service object.
- Convert interface/data format as needed.

Pros and Cons

Single Responsibility Principle. You can separate the interface or data conversion code from the primary business logic of the program.

Open/Closed Principle. You can introduce new types of adapters into the program without breaking the existing client code, as long as they work with the adapters through the client interface.

The overall complexity of the code increases because you need to introduce a set of new interfaces and classes. Sometimes it's **simpler just to change the service class** so that it matches the rest of your code.